Summarize

Chapter 4. Microservice Communication Styles

Getting communication between microservices right is problematic for many due in great part, I feel, to the fact that people gravitate toward a chosen technological approach without first considering the different types of communication they might want. In this chapter, I'll try and tease apart the different styles of communication to help you understand the pros and cons of each, as well as which approach will best fit your problem space.

We'll be looking at synchronous blocking and asynchronous nonblocking communication mechanisms, as well as comparing request-response collaboration with event-driven collaboration.

By the end of this chapter you should be much better prepared to understand the different options available to you and will have a foundational knowledge that will help when we start looking at more detailed implementation concerns in the following chapters.

From In-Process to Inter-Process

OK, let's get the easy stuff out of the way first—or at least what I *hope* is the easy stuff. Namely, calls *between* different processes across a network (inter-process) are *very* different from calls *within* a single process (in-process). At one level, we can ignore this distinction. It's easy, for example, to think of one object making a method call on another object and then just map this interaction to two microservices communicating via a network. Putting aside the fact that microservices aren't just objects, this thinking can get us into a lot of trouble.

Let's look at some of these differences and how they might change how you think about the interactions between your microservices.

Performance

The performance of an in-process call is fundamentally different from that of an inter-process call. When I make an in-process call, the underlying compiler and runtime can carry out a whole host of optimizations to reduce the impact of the call, including inlining the invocation so it's as though there was never a call in the first place. No such optimizations are possible with inter-process calls. Packets have to be sent. Expect the overhead of an inter-process call to be significant compared to the overhead of an in-process call. The former is very measurable—just round-tripping a single packet in a data center is measured in milliseconds—whereas the overhead of making a method call is something you don't need to worry about.

This can often lead you to want to rethink APIs. An API that makes sense in-process may not make sense in inter-process situations. I can make one thousand calls across an API boundary in-process without concern. Do I want to make one thousand network calls between two microservices? Perhaps not.

When I pass a parameter into a method, the data structure I pass in typically doesn't move—what's more likely is that I pass around a pointer to a memory location. Passing in an object or data structure to another method doesn't necessitate more memory being allocated in order to copy the data.

When making calls between microservices over a network, on the other hand, the data actually has to be serialized into some form that can be transmitted over a network. The data then needs to be sent and deserialized at the other end. We therefore may need to be more mindful about the size of payloads being sent between processes. When was the last time you were aware of the size of a data structure that you were passing around inside a process? The reality is that you likely didn't need to know; now you do. This might lead you to reduce the amount of data being sent or received (perhaps not a bad thing if we think about information hiding), pick more efficient serialization mechanisms, or even offload data to a filesystem and pass around a reference to that file location instead.

These differences may not cause you issues straight away, but you certainly need to be aware of them. I've seen a lot of attempts to hide from the developer the fact that a network call is even taking place. Our desire to create abstractions to hide detail is a big part of what allows us to do more things more efficiently, but sometimes we create abstractions that hide too much. A developer needs to be aware if they are doing something that will result in a network call; otherwise, you should not be surprised if you end

up with some nasty performance bottlenecks further down the line caused by odd inter-service interactions that weren't visible to the developer writing the code.

Changing Interfaces

When we consider changes to an interface inside a process, the act of rolling out the change is straightforward. The code implementing the interface and the code calling the interface are all packaged together in the same process. In fact, if I change a method signature using an IDE with refactoring capability, often the IDE itself will automatically refactor calls to this changing method. Rolling out such a change can be done in an atomic fashion—both sides of the interface are packaged together in a single process.

With communication between microservices, however, the microservice exposing an interface and the consuming microservices using that interface are separately deployable microservices. When making a backward-incompatible change to a microservice interface, we either need to do a lockstep deployment with consumers, making sure they are updated to use the new interface, or else find some way to phase the rollout of the new microservice contract. We'll explore this concept in more detail later in this chapter.

Error Handling

Within a process, if I call a method, the nature of the errors tends to be pretty straightforward. Simplistically, the errors either are expected and easy to handle, or they are catastrophic to the point that we just propagate the error up the call stack. Errors, on the whole, are deterministic.

With a distributed system, the nature of errors can be different. You are vulnerable to a host of errors that are outside your control. Networks time out. Downstream microservices might be temporarily unavailable. Networks get disconnected, containers get killed due to consuming too much memory, and in extreme situations, bits of your data center can catch fire.[1]

In their book *Distributed Systems*,[2] Andrew Tanenbaum and Maarten Steen break down the five types of failure modes you can see when looking at an inter-process communication. Here's a simplified version:

Crash failure

: Everything was fine till the server crashed. Reboot!

Omission failure

You sent something, but you didn't get a response. Also includes situations in which you expect a downstream microservice to be firing messages (perhaps including events), and it just stops.

Timing failure

Something happened too late (you didn't get it in time), or something happened too early!

Response failure

You got a response, but it just seems wrong. For example, you asked for an order summary, but needed pieces of information are missing in the response.

Arbitrary failure

Otherwise known as Byzantine failure, this is when something has gone wrong, but participants are unable to agree if the failure has occurred (or why). As it sounds like, this is a bad time all around.

Many of these errors are often transient in nature—they are short-lived problems that might go away. Consider the situation in which we send a request to a microservice but don't hear back (a type of omission failure). This could mean that the downstream microservice never got the request in the first place, so we need to send it again. Other problems can't be dealt with easily and might need a human operator to intervene. As a result, it can become important to have a richer set of semantics for returning errors in a way that can allow clients to take appropriate action.

HTTP is an example of a protocol that understands the importance of this. Every HTTP response has a code, with the 400 and 500 series codes being reserved for errors. 400 series error codes are request errors—essentially, a downstream service is telling the client that there is something wrong with the original request. As such, it's probably something you should give up on—is there any point retrying a `404 Not Found`, for example? The 500 series response codes relate to downstream issues, a subset of which indicate to the client that the issue might be temporary. A `503 Service Unavailable`, for example, indicates that the downstream server is unable to handle the request, but it may be a temporary state, in which case an upstream client might decide to retry the request. On the other hand, if a client receives a `501 Not Implemented` response, a retry is unlikely to help much.

Whether or not you pick an HTTP-based protocol for communication between microservices, if you have a rich set of semantics around the nature of the error, you'll make it

easier for clients to carry out compensating actions, which in turn should help you build more robust systems.

Technology for Inter-Process Communication: So Many Choices

> *And in a world where we have too many choices and too little time, the obvious thing to do is just ignore stuff.*
>
> —Seth Godin

The range of technology available to us for inter-process communication is vast. As a result, we can often be overburdened with choice. I often find that people gravitate to technology that is familiar to them, or perhaps just to the latest hot technology they learned about from a conference. The problem with this is that when you buy into a specific technology choice, you are often buying into a set of ideas and constraints that come along for the ride. These constraints might not be the right ones for you—and the mindset behind the technology may not actually line up with the problem you are trying to solve.

If you're trying to build a website, single page app technology like Angular or React is a bad fit. Likewise, trying to use Kafka for request-response really isn't a good idea, as it was designed for more event-based interactions (topics we'll get to in just a moment). And yet I see technology used in the wrong place time and time again. People pick the shiny new tech (like microservices!) without considering whether it actually fits their problem.

Thus when it comes to the bewildering array of technology available to us for communication between microservices, I think it's important to talk first about the style of communication you want, and only then look for the right technology to implement that style. With that in mind, let's take a look at a model I've been using for several years to help distinguish between the different approaches for microservice-to-microservice communication, which in turn can help you filter the technology options you'll want to look at.

Styles of Microservice Communication

In Figure 4-1 we see an outline for the model I use for thinking about different styles of communication. This model is not meant to be entirely exhaustive (I'm not trying to present a grand unified theory of inter-process communication here), but it provides a good high-level overview for considering the different styles of communication that are most widely used for microservice architectures.

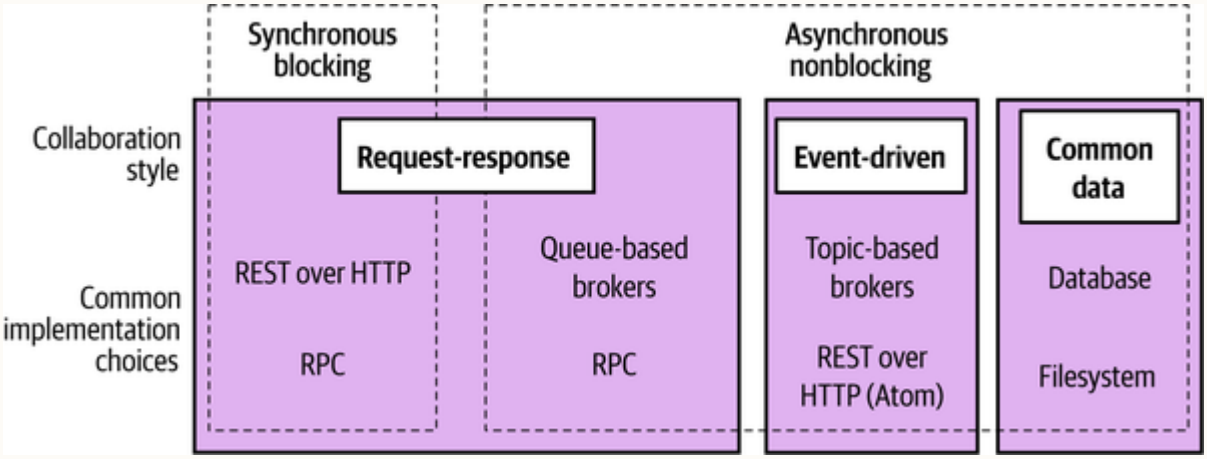


Figure 4-1. Different styles of inter-microservice communication along with example implementing technologies

We'll look at the different elements of this model in more detail shortly, but first I'd like to briefly outline them:

Synchronous blocking

A microservice makes a call to another microservice and blocks operation waiting for the response.

Asynchronous nonblocking

The microservice emitting a call is able to carry on processing whether or not the call is received.

Request-response

A microservice sends a request to another microservice asking for something to be done. It expects to receive a response informing it of the result.

Event-driven

Microservices emit events, which other microservices consume and react to accordingly. The microservice emitting the event is unaware of which microservices, if any, consume the events it emits.

Common data

> Not often seen as a communication style, microservices collaborate via some shared data source.

When using this model to help teams decide on the right approach, I spend a lot of time understanding the context in which they are operating. Their needs in terms of reliable communication, acceptable latency, and volume of communication are all going to play a part in making a technology choice. But in general, I tend to start with deciding whether a request-response or an event-driven style of collaboration is more appropriate for the given situation. If I'm looking at request-response, then both synchronous and asynchronous implementations are still available to me, so I have a second choice to make. If picking an event-driven collaboration style, however, my implementation choices will be limited to nonblocking asynchronous choices.

A host of other considerations come into play when picking the right technology that go beyond the style of communication—for example, the need for lower-latency communication, security-related aspects, or the ability to scale. It's unlikely that you can make a reasoned technology choice without taking into account the requirements (and constraints) of your specific problem space. When we look at technology options in Chapter 5, we'll discuss some of these issues.

Mix and Match

It's important to note that a microservice architecture as a whole may have a mix of styles of collaboration, and this is typically the norm. Some interactions just make sense as request-response, while others make sense as event-driven. In fact, it's common for a single microservice to implement more than one form of collaboration. Consider an `Order` microservice that exposes a request-response API that allows for orders to be placed or changed and then fires events when these changes are made.

With that said, let's look at these different styles of communication in more detail.

Pattern: Synchronous Blocking

With a synchronous blocking call, a microservice sends a call of some kind to a downstream process (likely another microservice) and blocks until the call has completed, and potentially until a response has been received. In Figure 4-2, the `Order Processor` sends a call to the `Loyalty` microservice to inform it that some points should be added to a customer's account.

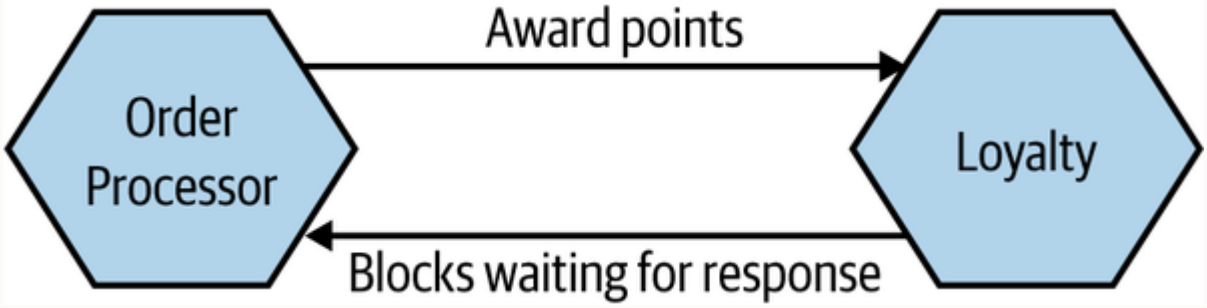


Figure 4-2. `Order Processor` sends a synchronous call to the `Loyalty` microservice, blocks, and waits for a response

Typically, a synchronous blocking call is one that is waiting for a response from the downstream process. This may be because the result of the call is needed for some further operation, or just because it wants to make sure the call worked and to carry out some sort of retry if not. As a result, virtually every synchronous blocking call I see would also constitute being a request-response call, something we'll look at shortly.

Advantages

There is something simple and familiar about a blocking, synchronous call. Many of us learned to program in a fundamentally synchronous style—reading a piece of code like a script, with each line executing in turn, and with the next line of code waiting its turn to do something. Most of the situations in which you would have used inter-process calls were probably done in a synchronous, blocking style—running a SQL query on a database, for example, or making an HTTP request of a downstream API.

When moving from a less distributed architecture, like that of a single process monolith, sticking with those ideas that are familiar when there is so much else going on that is brand-new can make sense.

Disadvantages

The main challenge with synchronous calls is the inherent temporal coupling that occurs, a topic we explored briefly in Chapter 2. When the `Order Processor` makes a call to `Loyalty` in the preceding example, the `Loyalty` microservice needs to be reachable for the call to work. If the `Loyalty` microservice is unavailable, then the call will fail, and `Order Processor` needs to work out what kind of compensating action to carry out—this might involve an immediate retry, buffering the call to retry later, or perhaps giving up altogether.

This coupling is two-way. With this style of integration, the response is typically sent over the same inbound network connection to the upstream microservice. So if the `Loyalty` microservice wants to send a response back to `Order Processor`, but the up-

stream instance has subsequently died, the response will get lost. The temporal coupling here isn't just between two microservices; it's between two specific instances of these microservices.

As the sender of the call is blocking and waiting for the downstream microservice to respond, it also follows that if the downstream microservice responds slowly, or if there is an issue with the latency of the network, then the sender of the call will be blocked for a prolonged period of time waiting for a response. If the `Loyalty` microservice is under significant load and is responding slowly to requests, this in turn will cause the `Order Processor` to respond slowly.

Thus the use of synchronous calls can make a system vulnerable to cascading issues caused by downstream outages more readily than can the use of asynchronous calls.

Where to Use It

For simple microservice architectures, I don't have a massive problem with the use of synchronous, blocking calls. Their familiarity for many people is an advantage when coming to grips with distributed systems.

For me, these types of calls begin to be problematic when you start having more chains of calls—in Figure 4-3, for instance, we have an example flow from MusicCorp, where we are checking a payment for potentially fraudulent activity. The `Order Processor` calls the `Payment` service to take payment. The `Payment` service in turn wants to check with the `Fraud Detection` microservice as to whether or not this should be allowed. The `Fraud Detection` microservice in turn needs to get information from the `Customer` microservice.

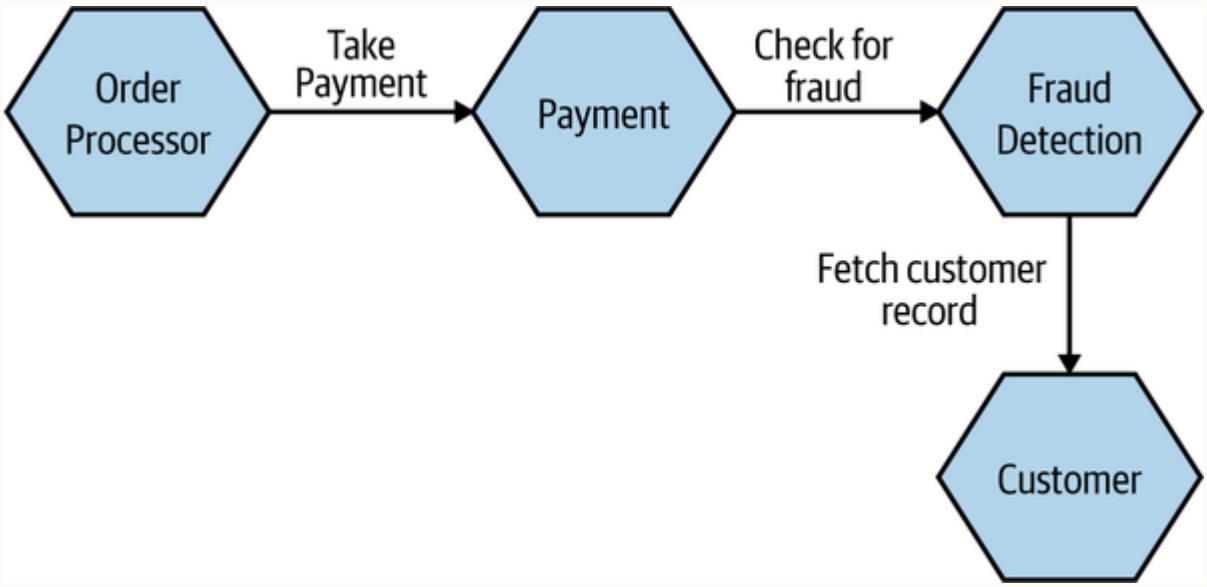


Figure 4-3. Checking for potentially fraudulent behavior as part of order processing flow

If all of these calls are synchronous and blocking, there are a number of issues we might face. An issue in any of the four involved microservices, or in the network calls between them, could cause the whole operation to fail. This is quite aside from the fact that these kinds of long chains can cause significant *resource contention*. Behind the scenes, the `Order Processor` likely has a network connection open waiting to hear back from `Payment`. `Payment` in turn has a network connection open waiting for a response from `Fraud Detection`, and so on. Having a lot of connections that need to be kept open can have an impact on the running system—you are much more likely to experience issues in which you run out of available connections or suffer from increased network congestion as a result.

To improve this situation, we could reexamine the interactions between the microservices in the first place. For example, maybe we take the use of `Fraud Detection` out of the main purchase flow, as shown in Figure 4-4, and instead have it run in the background. If it finds a problem with a specific customer, their records are updated accordingly, and this is something that could be checked earlier in the payment process. Effectively, this means we're doing some of this work in parallel. By reducing the length of the call chain, we'll see the overall latency of the operation improve, and we'll take one of our microservices (`Fraud Detection`) out of the critical path for the purchase flow, giving us one fewer dependency to worry about for what is a critical operation.

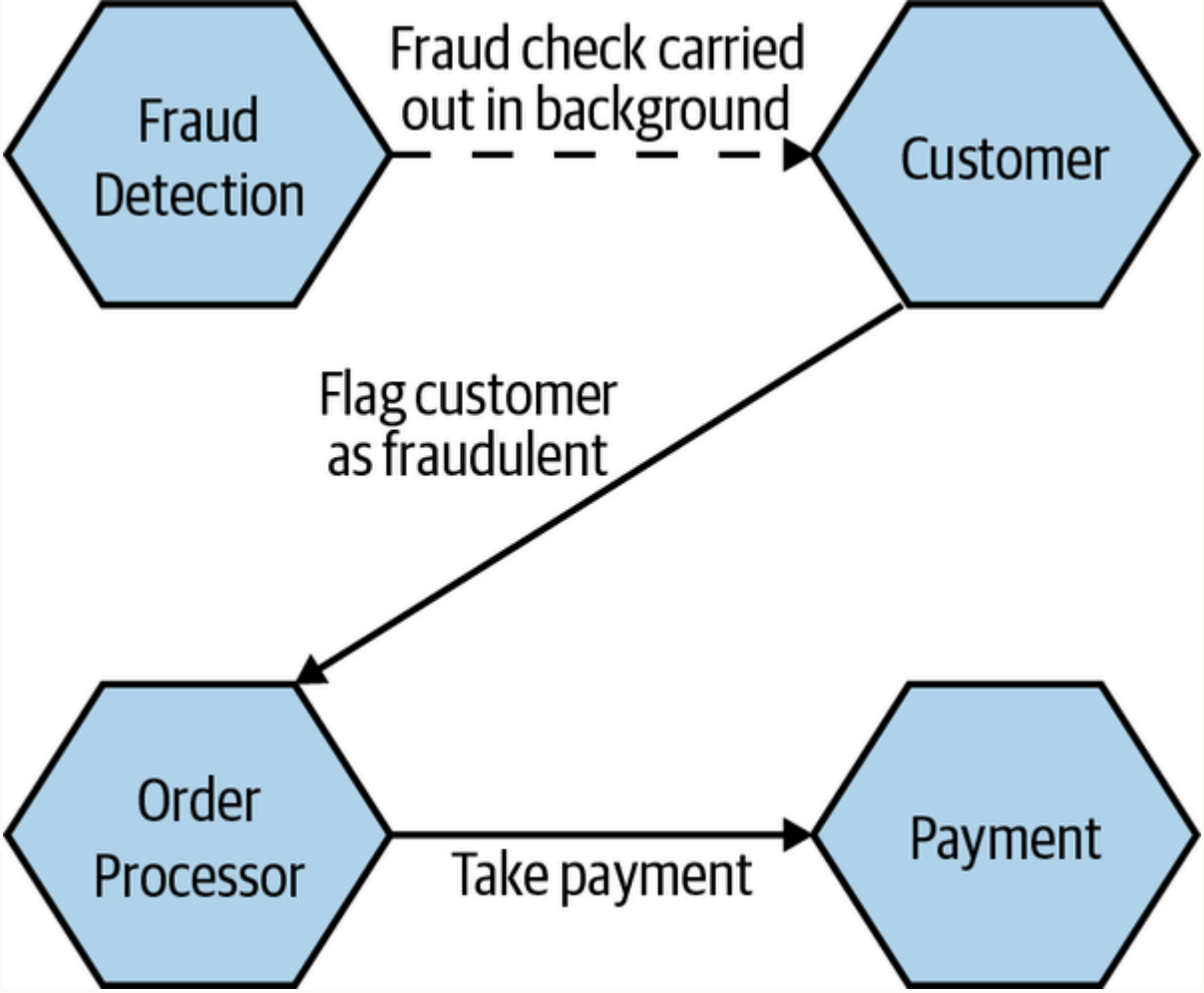


Figure 4-4. Moving `Fraud Detection` to a background process can reduce concerns around the length of the call chain

Of course, we could also replace the use of blocking calls with some style of nonblocking interaction without changing the workflow here, an approach we'll explore next.

Pattern: Asynchronous Nonblocking

With asynchronous communication, the act of sending a call out over the network doesn't block the microservice issuing the call. It is able to carry on with any other processing without having to wait for a response. Nonblocking asynchronous communication comes in many forms, but we'll be looking in more detail at the three most common styles I see in microservice architecture. They are:

Communication through common data

The upstream microservice changes some common data, which one or more microservices later make use of.

Request-response

A microservice sends a request to another microservice asking it to do something. When the requested operation completes, whether successfully or not, the up-

stream microservice receives the response. Specifically, *any* instance of the upstream microservice should be able to handle the response.

Event-driven interaction

A microservice broadcasts an event, which can be thought of as a factual statement about something that has happened. Other microservices can listen for the events they are interested in and react accordingly.

Advantages

With nonblocking asynchronous communication, the microservice making the initial call and the microservice (or microservices) receiving the call are decoupled temporally. The microservices that receive the call do not need to be reachable at the same time the call is made. This means we avoid the concerns of temporal decoupling that we discussed in Chapter 2 (see "A Brief Note on Temporal Coupling").

This style of communication is also beneficial if the functionality being triggered by a call will take a long time to process. Let's come back to our example of MusicCorp, and specifically the process of sending out a package. In Figure 4-5, the `Order Processor` has taken payment and has decided that it is time to dispatch the package, so it sends a call to the `Warehouse` microservice. The process of finding the CDs, taking them off the shelf, packaging them up, and having them picked up could take many hours, and potentially even days, depending on how the actual dispatch process works. It makes sense, therefore, for the `Order Processor` to issue a nonblocking asynchronous call to the `Warehouse` and have the `Warehouse` call back later to inform the `Order Processor` of its progress. This is a form of asynchronous request-response communication.

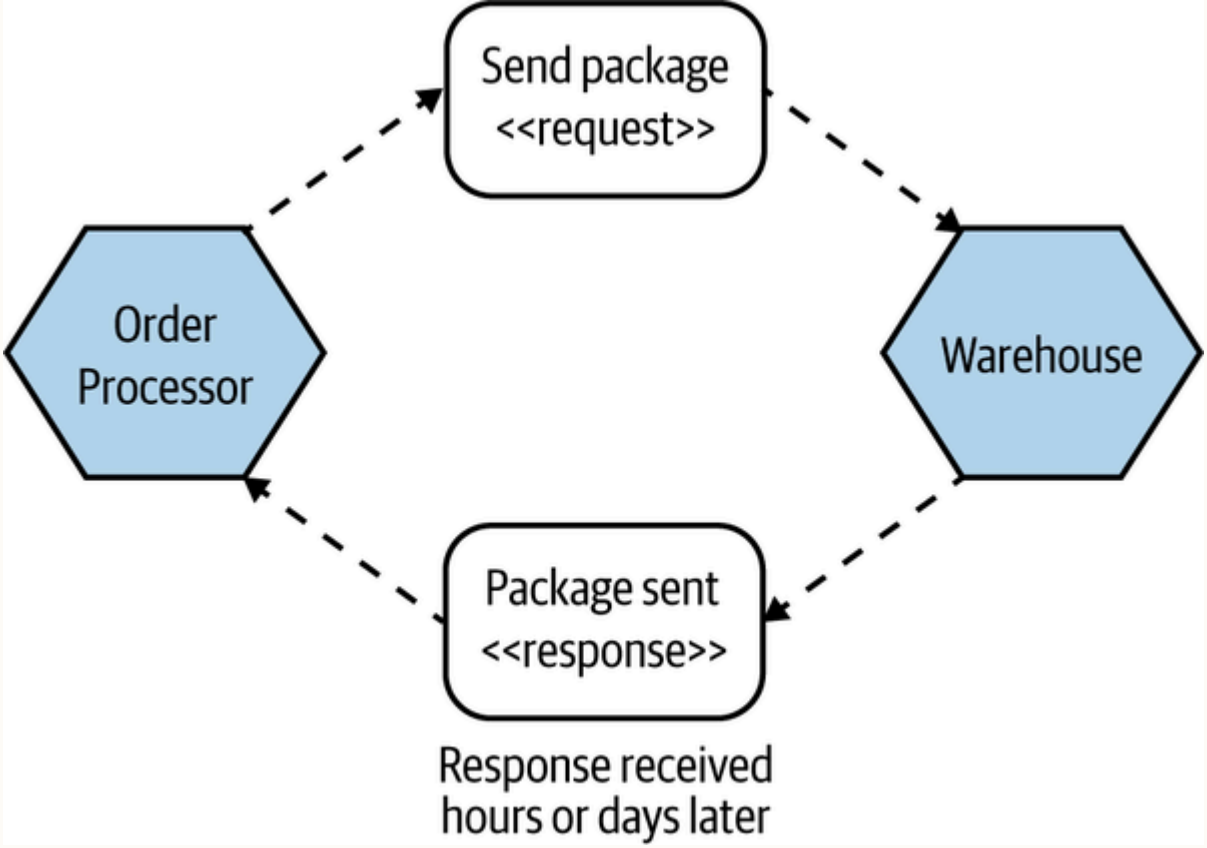


Figure 4-5. The `Order Processor` kicks off the process to package and ship an order, which is done in an asynchronous fashion

If we tried doing something similar with synchronous blocking calls, then we'd have to restructure the interactions between `Order Processor` and `Warehouse`—it wouldn't be feasible for `Order Processor` to open a connection, send a request, block any further operations in calling the thread, and wait for a response for what might be hours or days.

Disadvantages

The main downsides of nonblocking asynchronous communication, relative to blocking synchronous communication, are the level of complexity and the range of choice. As we've already outlined, there are different styles of asynchronous communication to choose from—which one is right for you? When we start digging into how these different styles of communication are implemented, there is a potentially bewildering list of technology we could look at.

If asynchronous communication doesn't map to your mental models of computing, adopting an asynchronous style of communication will be challenging at first. And as we'll explore further when we look in detail at the various styles of asynchronous communication, there are a lot of different, interesting ways in which you can get yourself into a *lot* of trouble.

ASYNC/AWAIT AND WHEN ASYNCHRONOUS IS STILL BLOCKING

As with many areas of computing, we can use the same term in different contexts to have very different meanings. A style of programming that appears to be especially popular is the use of constructs like async/await to work with a potentially asynchronous source of data but in a blocking, synchronous style.

In Example 4-1, we see a very simple JavaScript example of this in action. The currency exchange rates fluctuate frequently through the day, and we receive these via a message broker. We define a `Promise`. Generically, a promise is something that will resolve to a state at some point in the future. In our case, our `eurToGbp` will eventually resolve to being the next euro-to-GBP exchange rate.

Example 4-1. An example of working with a potentially asynchronous call in a blocking, synchronous fashion

```
async function f() {

  let eurToGbp = new Promise((resolve, reject) => {
    //code to fetch latest exchange rate between EUR and GBP
    ...
  });

  var latestRate = await eurToGbp; ❶
  process(latestRate); ❷
}
```

❶ Wait until the latest EUR-to-GBP exchange rate is fetched.

❷ Won't run until the promise is fulfilled.

When we reference `eurToGbp` using `await`, we block until `latestRate`'s state is fulfilled — `process` isn't reached until we resolve the state of `eurToGbp`.[3]

Even though our exchange rates are being received in an asynchronous fashion, the use of `await` in this context means we are *blocking* until the state of `latestRate` is resolved. So even if the underlying technology we are using to get the rate could be considered asynchronous in nature (for example, waiting for the rate), from the point of our code, this is inherently a synchronous, blocking interaction.

Where to Use It

Ultimately, when considering whether asynchronous communication is right for you, you also have to consider which *type* of asynchronous communication you want to pick, as each type has its own trade-offs. In general, though, there are some specific use cases that would have me reaching for some form of asynchronous communication. Long-running processes are an obvious candidate, as we explored in Figure 4-5. Also, situations in which you have long call chains that you can't easily restructure could be a good candidate. We'll dive deeper into this when we look at three of the most common forms of asynchronous communication—request-response calls, event-driven communication, and communication through common data.

Pattern: Communication Through Common Data

A style of communication that spans a multitude of implementations is communication through common data. This pattern is used when one microservice puts data into a defined location, and another microservice (or potentially multiple microservices) then makes use of the data. It can be as simple as one microservice dropping a file in a location, and at some point later on another microservice picks up that file and does something with it. This integration style is fundamentally asynchronous in nature.

An example of this style can be seen in Figure 4-6, where the `New Product Importer` creates a file that will then be read by the downstream `Inventory` and `Catalog` microservices.

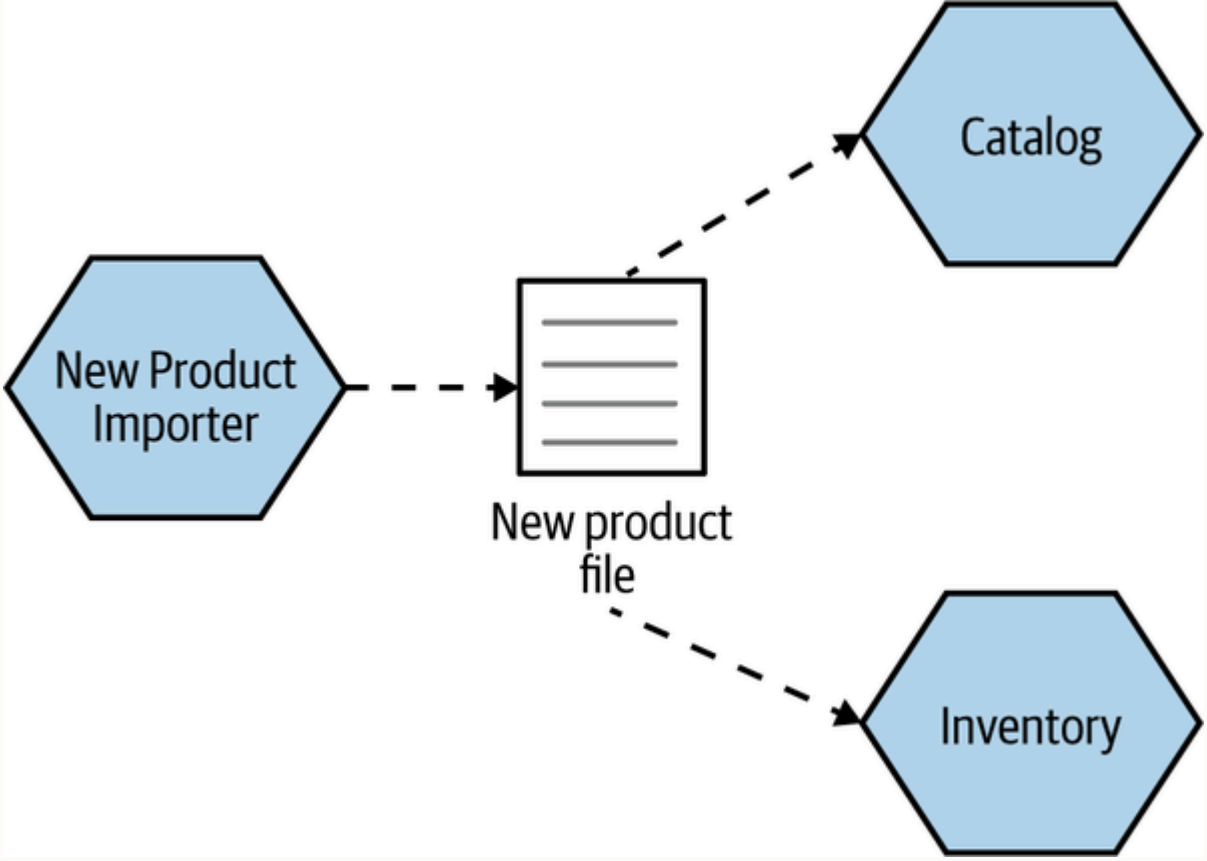


Figure 4-6. One microservice writes out a file that other microservices make use of

This pattern is in some ways the most common general inter-process communication pattern that you'll see, and yet we sometimes fail to see it as a communication pattern at all—I think largely because the communication between processes is often so indirect as to be hard to spot.

Implementation

To implement this pattern, you need some sort of persistent store for the data. A filesystem in many cases can be enough. I've built many systems that just periodically scan a filesystem, note the presence of a new file, and react on it accordingly. You could use some sort of robust distributed memory store as well, of course. It's worth noting that any downstream microservice that is going to act on this data will need its own mechanism to identify that new data is available—polling is a frequent solution to this problem.

Two common examples of this pattern are the data lake and the data warehouse. In both cases, these solutions are typically designed to help process large volumes of data, but arguably they exist at opposite ends of the spectrum regarding coupling. With a data lake, sources upload raw data in whatever format they see fit, and downstream consumers of this raw data are expected to know how to process the information. With a data warehouse, the warehouse itself is a structured data store. Microservices pushing data to the data warehouse need to know the structure of the data warehouse—if the

structure changes in a backward-incompatible way, then these producers will need to be updated.

With both the data warehouse and the data lake, the assumption is that the flow of information is in a single direction. One microservice publishes data to the common data store, and downstream consumers read that data and carry out appropriate actions. This unidirectional flow can make it easier to reason about the flow of information. A more problematic implementation would be the use of a shared database in which multiple microservices both read and write to the same data store, an example of which we discussed in Chapter 2 when we explored common coupling—Figure 4-7 shows both `Order Processor` and `Warehouse` updating the same record.

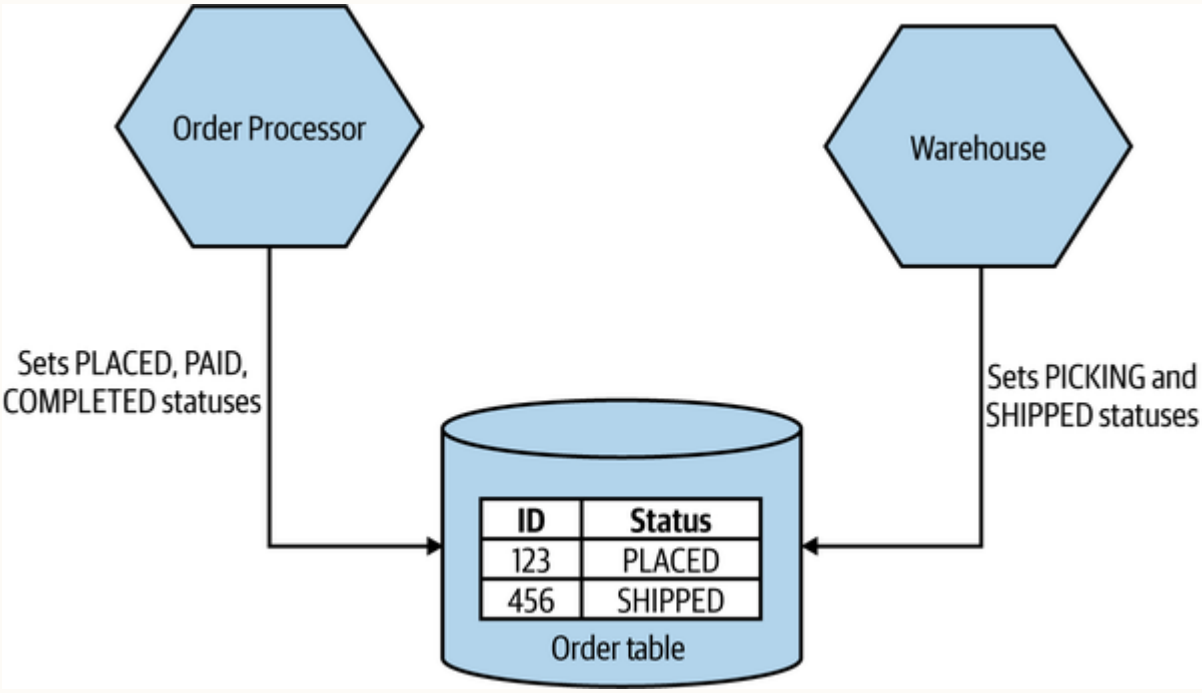


Figure 4-7. An example of common coupling in which both `Order Processor` and `Warehouse` are updating the same order record

Advantages

This pattern can be implemented very simply, using commonly understood technology. If you can read or write to a file or read and write to a database, you can use this pattern. The use of prevalent and well-understood technology also enables interoperability between different types of systems, including older mainframe applications or customizable off-the-shelf (COTS) software products. Data volumes are also less of a concern here—if you're sending lots of data in one big go, this pattern can work well.

Disadvantages

Downstream consuming microservices will typically be aware that there is new data to process via some sort of polling mechanism, or else perhaps through a periodically triggered timed job. That means that this mechanism is unlikely to be useful in low-latency situations. You can of course combine this pattern with some other sort of call informing a downstream microservice that new data is available. For example, I could write a file to a shared filesystem and then send a call to the interested microservice informing it that there is new data that it may want. This can close the gap between data being published and data being processed. In general, though, if you're using this pattern for very large volumes of data, it's less likely that low latency is high on your list of requirements. If you are interested in sending larger volumes of data and having them processed more in "real time," then using some sort of streaming technology like Kafka would be a better fit.

Another big disadvantage, and something that should be fairly obvious if you remember back to our exploration of common coupling in Figure 4-7, is that the common data store becomes a potential source of coupling. If that data store changes structure in some way, it can break communication between microservices.

The robustness of the communication will also come down to the robustness of the underlying data store. This isn't a disadvantage strictly speaking, but it's something to be aware of. If you're dropping a file on a filesystem, you might want to make sure that the filesystem itself isn't going to fail in interesting ways.

Where to Use It

Where this pattern really shines is in enabling interoperability between processes that might have restrictions on what technology they can use. Having an existing system talk to your microservice's GRPC interface or subscribe to its Kafka topic might well be more convenient from the point of view of the microservice, but not from the point of view of a consumer. Older systems may have limitations on what technology they can support and may have high costs of change. On the other hand, even old mainframe systems should be able to read data out of a file. This does of course all depend on using data store technology that is widely supported—I could also implement this pattern using something like a Redis cache. But can your old mainframe system talk to Redis?

Another major sweet spot for this pattern is in sharing large volumes of data. If you need to send a multigigabyte file to a filesystem or load a few million rows into a database, then this pattern is the way to go.

Pattern: Request-Response Communication

With request-response, a microservice sends a request to a downstream service asking it to do something and expects to receive a response with the result of the request. This interaction can be undertaken via a synchronous blocking call, or it could be implemented in an asynchronous nonblocking fashion. A simple example of this interaction is shown in Figure 4-8, where the `Chart` microservice, which collates the best-selling CDs for different genres, sends a request to the `Inventory` service asking for the current stock levels for some CDs.

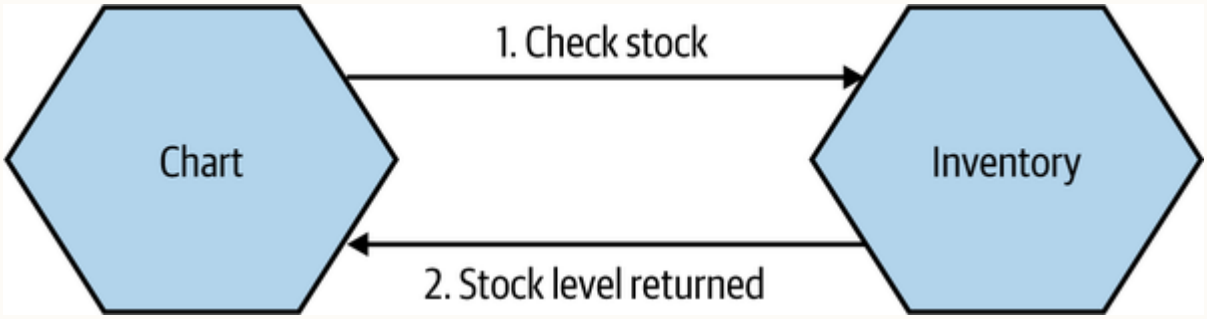


Figure 4-8. The `Chart` microservice sends a request to `Inventory` asking for stock levels

Retrieving data from other microservices like this is a common use case for a request-response call. Sometimes, though, you just need to make sure something gets done. In Figure 4-9, the `Warehouse` microservice is sent a request from `Order Processor` asking it to reserve stock. The `Order Processor` just needs to know that stock has been successfully reserved before it can carry on with taking payment. If the stock can't be reserved—perhaps because an item is no longer available—then the payment can be cancelled. Using request-response calls in situations like this where calls need to be completed in a certain order is commonplace.

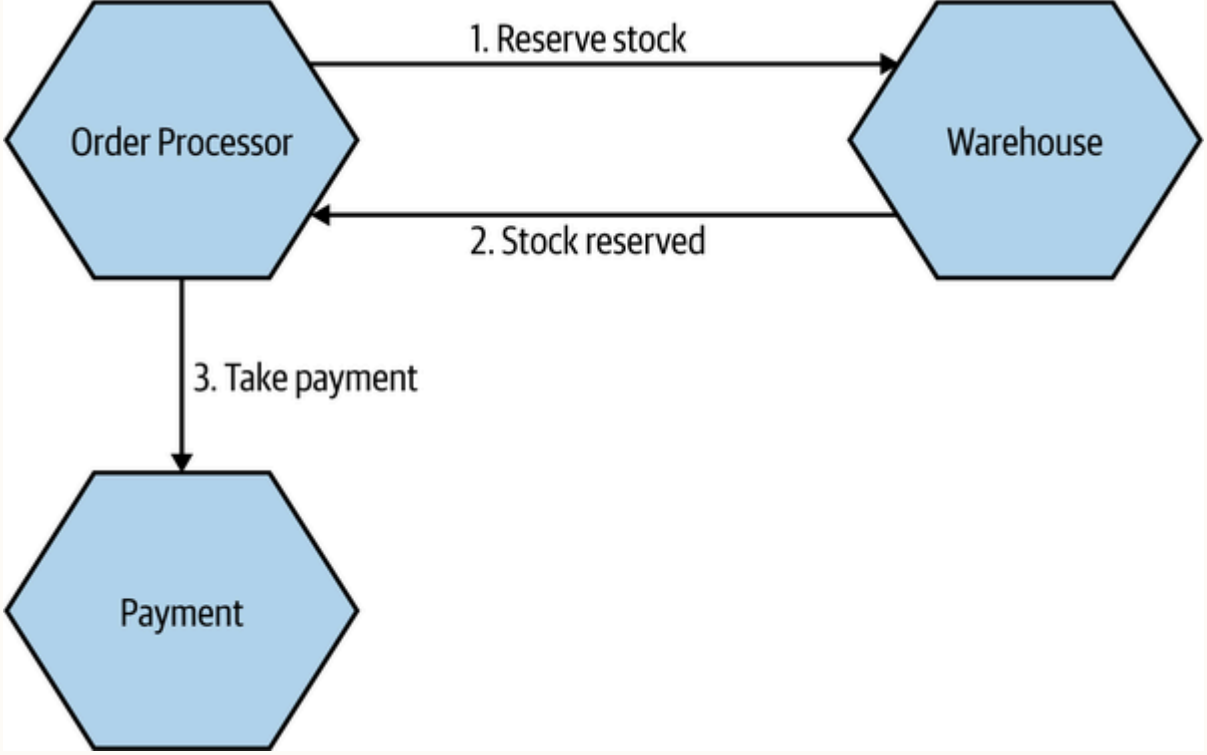


Figure 4-9. `Order Processor` needs to ensure stock can be reserved before payment can be taken

---

COMMANDS VERSUS REQUESTS

I've heard some people talk about sending commands, rather than requests, specifically in the context of asynchronous request-response communication. The intent behind the term *command* is arguably the same as that of *request*—namely, an upstream microservice is asking a downstream microservice to do something.

Personally speaking, though, I much prefer the term *request*. A command implies a directive that must be obeyed, and it can lead to a situation in which people feel that the command has to be acted on. A request implies something that can be rejected. It is right that a microservice examines each request on its merits and, based on its own internal logic, decides whether the request should be acted on. If the request it has been sent violates internal logic, the microservice should reject it. Although it's a subtle difference, I don't feel that the term *command* conveys the same meaning.

I'll stick to using *request* over *command*, but whatever term you decide to use, just remember that a microservice gets to reject the request/command if appropriate.

Implementation: Synchronous Versus Asynchronous

Request-response calls like this can be implemented in either a blocking synchronous or a nonblocking asynchronous style. With a synchronous call, what you'd typically see is a network connection being opened with the downstream microservice, with the request

being sent along this connection. The connection is kept open while the upstream microservice waits for the downstream microservice to respond. In this case, the microservice sending the response doesn't really need to know anything about the microservice that sent the request—it's just sending stuff back over an inbound connection. If that connection dies, perhaps because either the upstream or the downstream microservice instance dies, then we might have a problem.

With an asynchronous request-response, things are less straightforward. Let's revisit the process associated with reserving stock. In Figure 4-10, the request to reserve stock is sent as a message over some sort of message broker (we'll explore message brokers later in this chapter). Rather than the message going directly to the `Inventory` microservice from `Order Processor`, it instead sits in a queue. The `Inventory` consumes messages from this queue when it is able. It reads the request, carries out the associated work of reserving the stock, and then needs to send the response back to a queue that `Order Processor` is reading from. The `Inventory` microservice needs to know where to route the response. In our example, it sends this response back over another queue that is in turn consumed by `Order Processor`.

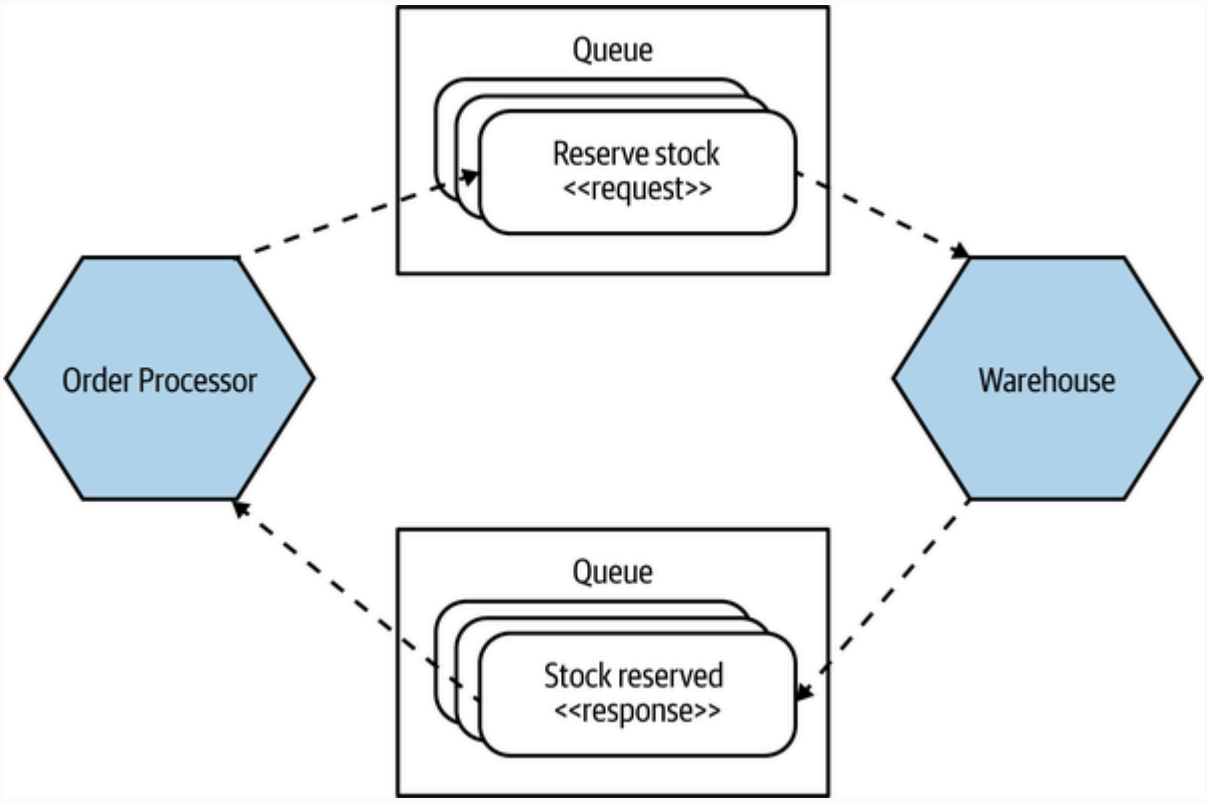


Figure 4-10. Using queues to send stock reservation requests

So with a nonblocking asynchronous interaction, the microservice that receives the request needs either to know implicitly where to route the response or else to be told where the response should go. When using a queue, we have the added benefit that multiple requests could be buffered up in the queue waiting to be handled. This can help in situations in which the requests can't be handled quickly enough. The microservice can

consume the next request when it is ready, rather than being overwhelmed by too many calls. Of course, a lot then depends on the queue absorbing these requests.

When a microservice receives a response in this way, it might need to relate the response to the original request. This can be challenging, as a lot of time may have passed, and depending on the nature of the protocol being used, the response may not come back to the same instance of the microservice that sent the request. In our example of reserving stock as part of placing an order, we'd need to know how to associate the "stock reserved" response with a given order so we can carry on processing that particular order. An easy way to handle this would be to store any state associated with the original request into a database, such that when the response comes in, the receiving instance can reload any associated state and act accordingly.

One last note: all forms of request-response interaction are likely going to require some form of time-out handling to avoid issues where the system gets blocked waiting for something that may never happen. How this time-out functionality is implemented can vary based on the implementation technology, but it will be needed. We'll look at time-outs in more detail in Chapter 12.

PARALLEL VERSUS SEQUENTIAL CALLS

When working with request-response interactions, you'll often encounter a situation in which you will need to make multiple calls before you can continue with some processing.

Consider a situation in which MusicCorp needs to check on the price for a given item from three different stockists, which we do by issuing API calls. We want to get the prices back from all three stockists before deciding which one we want to order new stock from. We could decide to make the three calls in sequence—waiting for each one to finish before proceeding with the next. In such a situation, we'd be waiting for the sum of latencies of each of the calls. If the API call to each provider took one second to return, we'd be waiting three seconds before we can decide who we should order from.

A better option would be to run these three requests in parallel; then the overall latency of the operation would be based on the slowest API call, rather than on the sum of latencies of each API call.

Reactive extensions and mechanisms like async/await can be very useful in helping to run calls in parallel, and this can result in significant improvements in the latency of some operations.

Where to Use It

Request-response calls make perfect sense for any situation in which the result of a request is needed before further processing can take place. They also fit really well in situations where a microservice wants to know if a call didn't work so that it can carry out some sort of compensating action, like a retry. If either aligns with your situation, then request-response is a sensible approach; the only remaining question is to decide on a synchronous versus asynchronous implementation, with the same trade-offs we discussed earlier.

Pattern: Event-Driven Communication

Event-driven communication looks quite odd compared to request-response calls. Rather than a microservice asking some other microservice to do something, a microservice emits events that may or may not be received by other microservices. It is an inherently asynchronous interaction, as the event listeners will be running on their own thread of execution.

An event is a statement about something that has occurred, nearly always something that has happened inside the world of the microservice that is emitting the event. The microservice emitting the event has no knowledge of the intent of other microservices to use the event, and indeed it may not even be aware that other microservices exist. It emits the event when required, and that is the end of its responsibilities.

In Figure 4-11, we see the `Warehouse` emitting events related to the process of packaging up an order. These events are received by two microservices, `Notifications` and `Inventory`, and they react accordingly. The `Notifications` microservice sends an email to update our customer about changes in order status, while the `Inventory` microservice can update stock levels as items are packaged into the customer's order.

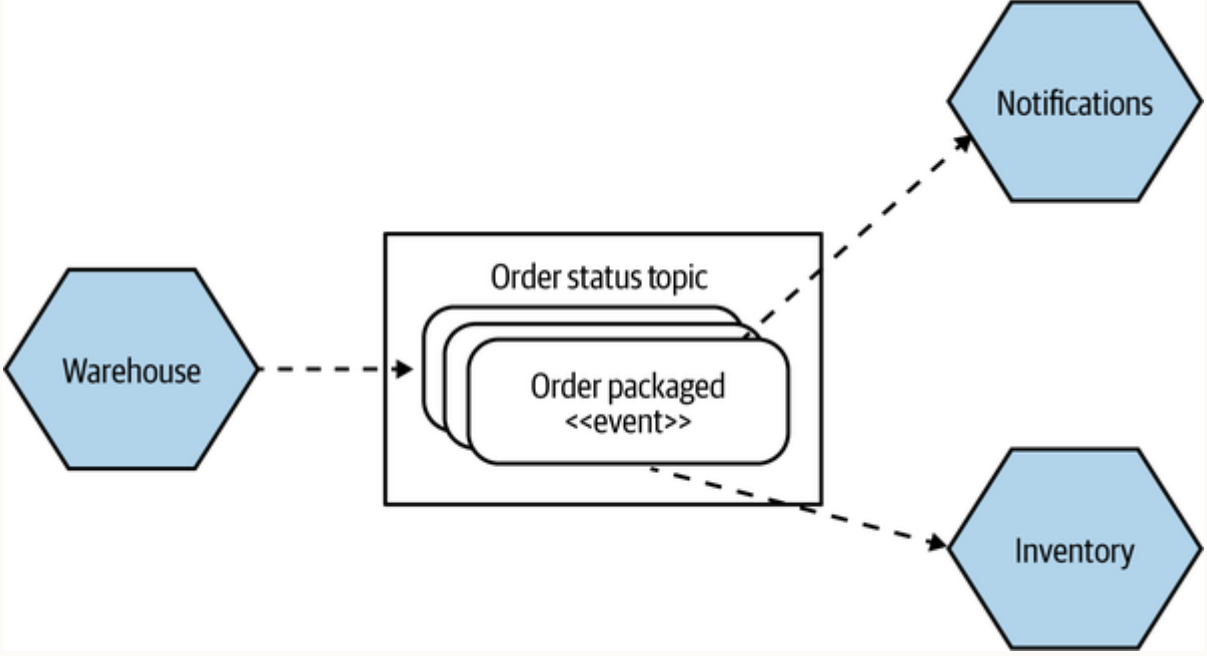


Figure 4-11. The `Warehouse` emits events that some downstream microservices subscribe to

The `Warehouse` is just broadcasting events, assuming that interested parties will react accordingly. It is unaware of who the recipients of the events are, making event-driven interactions much more loosely coupled in general. When you compare this to a request-response call, it may take you a while to get your head around the inversion of responsibility. With request-response, we might instead expect `Warehouse` to tell the `Notifications` microservice to send emails when appropriate. In such a model, `Warehouse` would need to know what events require customer notification. With an event-driven interaction, we are instead pushing that responsibility into the `Notifications` microservice.

The intent behind an event could be considered the opposite of a request. The event emitter is leaving it up to the recipients to decide what to do. With request-response, the microservice sending the request knows what should be done and is telling the other microservice what it thinks needs to happen next. This of course means that in request-response, the requester has to have knowledge of what the downstream recipient can do, implying a greater degree of domain coupling. With event-driven collaboration, the event emitter doesn't need to know what any downstream microservices are able to do, and in fact may not even know they exist—as a result, coupling is greatly reduced.

The distribution of responsibility we see with our event-driven interactions can mirror the distribution of responsibility we see with organizations trying to create more autonomous teams. Rather than holding all the responsibility centrally, we want to push it into the teams themselves to allow them to operate in a more autonomous fashion—a concept we will revisit in Chapter 15. Here, we are pushing responsibility from `Warehouse` into `Notifications` and `Inventory`—this can help us reduce the complexity of microservices like `Warehouse` and lead to a more even distribution of "smarts" in our system.

We'll explore that idea in more detail when we compare choreography and orchestration in Chapter 6.

EVENTS AND MESSAGES

On occasion I've seen the terms *messages* and *events* get confused. An event is a fact—a statement that something happened, along with some information about exactly what happened. A message is a thing we send over an asynchronous communication mechanism, like a message broker.

With event-driven collaboration, we want to broadcast that event, and a typical way to implement that broadcast mechanism would be to put the event into a message. The message is the medium; the event is the payload.

Likewise, we might want to send a request as the payload of a message—in which case we would be implementing a form of asynchronous request-response.

Implementation

There are two main aspects we need to consider here: a way for our microservices to emit events, and a way for our consumers to find out those events have happened.

Traditionally, message brokers like RabbitMQ try to handle both problems. Producers use an API to publish an event to the broker. The broker handles subscriptions, allowing consumers to be informed when an event arrives. These brokers can even handle the state of consumers—for example, by helping keep track of what messages they have seen before. These systems are normally designed to be scalable and resilient, but that doesn't come for free. It can add complexity to the development process, because it is another system you may need to run to develop and test your services. Additional machines and expertise may also be required to keep this infrastructure up and running. But once it is, it can be an incredibly effective way to implement loosely coupled, event-driven architectures. In general, I'm a fan.

Do be wary, though, about the world of middleware, of which the message broker is just a small part. Queues in and of themselves are perfectly sensible, useful things. However, vendors tend to want to package lots of software with them, which can lead to more and more smarts being pushed into the middleware, as evidenced by things like the enterprise service bus. Make sure you know what you're getting: keep your middleware dumb, and keep the smarts in the endpoints.

Another approach is to try to use HTTP as a way of propagating events. Atom is a REST-compliant specification that defines semantics (among other things) for publishing feeds of resources. Many client libraries exist that allow us to create and consume these feeds. So our customer service could just publish an event to such a feed whenever our customer service changes. Our consumers simply poll the feed, looking for changes. On one hand, the fact that we can reuse the existing Atom specification and any associated libraries is useful, and we know that HTTP handles scale very well. However, this use of HTTP is not good at low latency (where some message brokers excel), and we still must deal with the fact that the consumers need to keep track of what messages they have seen and manage their own polling schedule.

I have seen people spend ages implementing more and more of the behaviors that you get out of the box with an appropriate message broker to make Atom work for some use cases. For example, the *competing consumer pattern* describes a method whereby you bring up multiple worker instances to compete for messages, which works well for scaling up the number of workers to handle a list of independent jobs (we'll come back to that in the next chapter). However, we want to avoid the case in which two or more workers see the same message, as we'll end up doing the same task more than we need to. With a message broker, a standard queue will handle this. With Atom, we now need to manage our own shared state among all the workers to try to reduce the chances of reproducing effort.

If you already have a good, resilient message broker available to you, consider using it to handle publishing and subscribing to events. If you don't already have one, give Atom a look, but be aware of the sunk cost fallacy. If you find yourself wanting more and more of the support that a message broker gives you, at a certain point you might want to change your approach.

In terms of what we actually send over these asynchronous protocols, the same considerations apply as with synchronous communication. If you are currently happy with encoding requests and responses using JSON, stick with it.

What's in an Event?

In Figure 4-12, we see an event being broadcast from the `Customer` microservice, informing interested parties that a new customer has registered with the system. Two of the downstream microservices, `Loyalty` and `Notifications`, care about this event. The `Loyalty` microservice reacts to receiving the event by setting up an account for the new customer so that they can start earning points, whereas the `Notifications` microservice

sends an email to the newly registered customer welcoming them to the wondrous delights of MusicCorp.

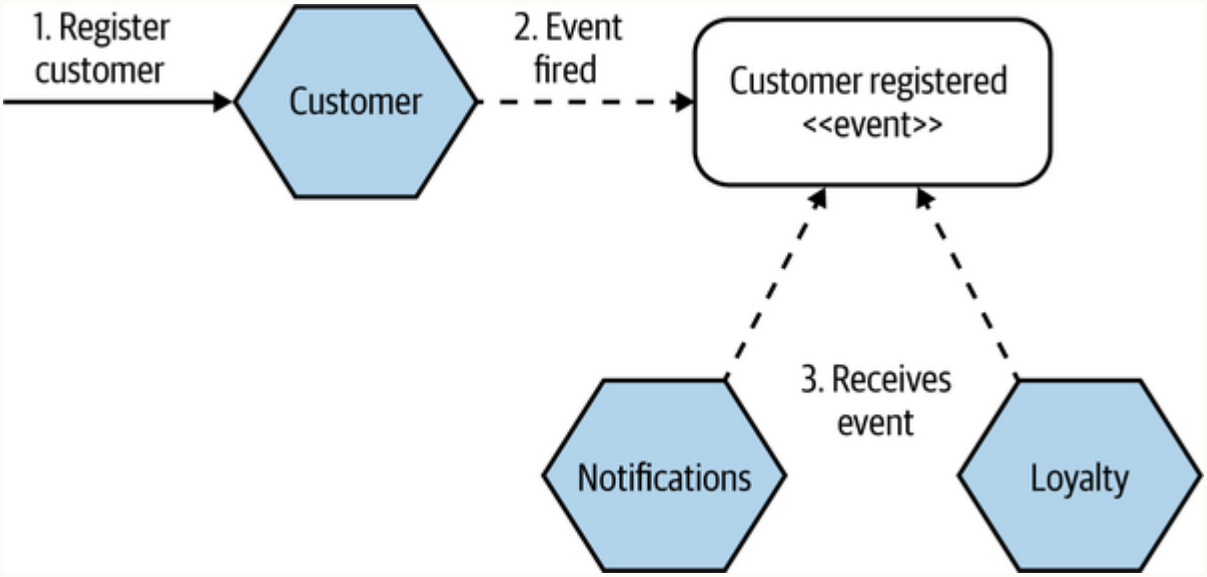


Figure 4-12. `Notifications` and `Loyalty` microservices receive an event when a new customer is registered

With a request, we are asking a microservice to do something and providing the required information for the requested operation to be carried out. With an event, we are broadcasting a fact that other parties *might* be interested in, but as the microservice emitting an event can't and shouldn't know who receives the event, how do we know what information other parties might need from the event? What, exactly, should be inside the event?

Just an ID

One option is for the event to just contain an identifier for the newly registered customer, as shown in Figure 4-13. The `Loyalty` microservice needs only this identifier to create the matching loyalty account, so it has all the information it needs. However, while the `Notifications` microservice knows that it needs to send a welcome email when this type of event is received, it will need additional information to do its job—at least an email address, and probably the name of the customer as well to give the email that personal touch. As this information isn't in the event that the `Notifications` microservice receives, it has no choice but to fetch this information from the `Customer` microservice, something we see in Figure 4-13.

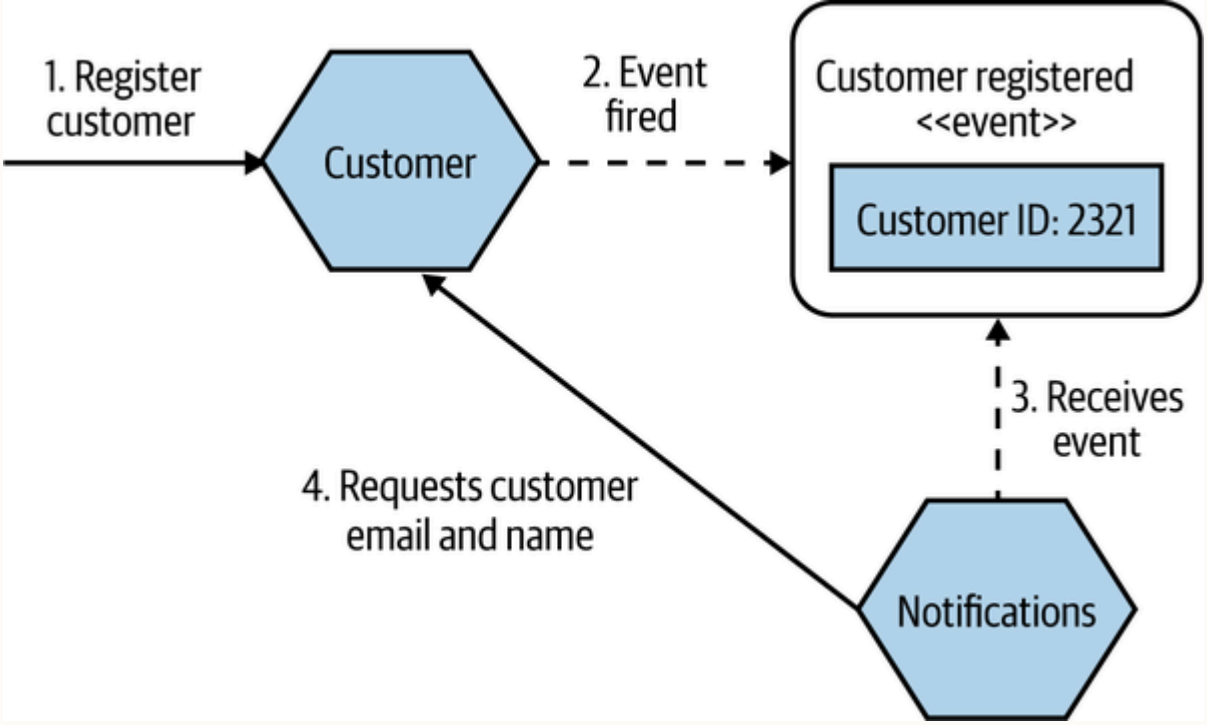


Figure 4-13. The `Notifications` microservice needs to request further details from the `Customer` microservice that aren't included in the event

There are some downsides with this approach. Firstly, the `Notifications` microservice now has to know about the `Customer` microservice, adding additional domain coupling. While domain coupling, as we discussed in Chapter 2, is on the looser end of the coupling spectrum, we'd still like to avoid it where possible. If the event that the `Notification`s microservice received contained all the information it needed, then this callback wouldn't be required. The callback from the receiving microservice can also lead to the other major downside—namely, that in a situation with a large number of receiving microservices, the microservice emitting the event might get a barrage of requests as a result. Imagine if five different microservices all received the same customer creation event, and all needed to request additional information—they'd all have to immediately send a request to the `Customer` microservice to get what they needed. As the number of microservices interested in a particular event increases, the impact of these calls could become significant.

Fully detailed events

The alternative, which I prefer, is to put everything into an event that you would be happy otherwise sharing via an API. If you'd let the `Notifications` microservice ask for the email address and name of a given customer, why not just put that information in the event in the first place? In Figure 4-14, we see this approach— `Notification`s is now more self-sufficient and is able to do its job without needing to communicate with the `Customer` microservice. In fact, it might never need to know the `Customer` microservice exists.

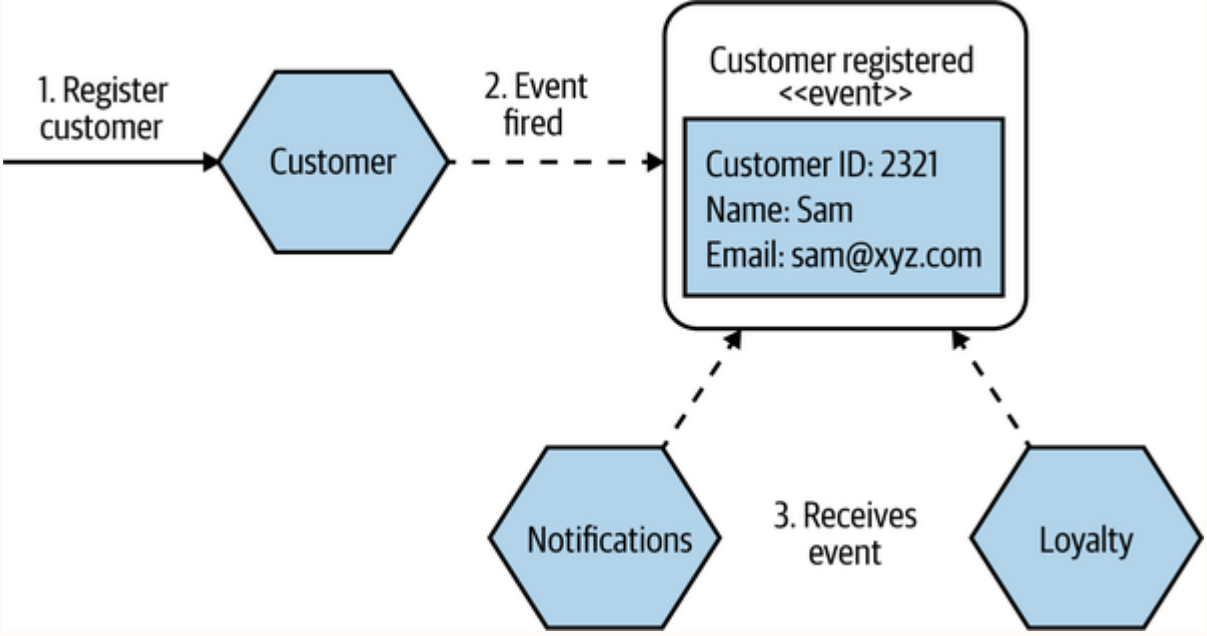


Figure 4-14. An event with more information in it can allow receiving microservices to act without requiring further calls to the source of the event

In addition to the fact that events with more information can allow for looser coupling, events with more information can also double as a historical record of what happened to a given entity. This could help you as part of implementing an auditing system, or perhaps even provide the ability to reconstitute an entity at given points in time—meaning that these events could be used as part of an event sourcing, a concept we'll explore briefly in a moment.

While this approach is definitely my preference, it's not without downsides. Firstly, if the data associated with an event is large, we might have concerns about the size of the event. Modern message brokers (assuming you're using one to implement your event broadcast mechanism) have fairly generous limits for message size; the default maximum size for a message in Kafka is 1 MB, and the latest release of RabbitMQ has a theoretical upper limit of 512 MB for a single message (down from the previous limit of 2 GB!), even though one could expect there to be some interesting performance issues with large messages like this. But even the 1 MB afforded to us as the maximum size of a message on Kafka gives us a lot of scope to send quite a bit of data. Ultimately, if you're venturing into a space in which you are starting to worry about the size of your events, then I'd recommend a hybrid approach in which some information is in the event but other (larger) data can be looked up if required.

In Figure 4-14, `Loyalty` doesn't need to know the email address or name of the customer, and yet it nonetheless receives it via the event. This could lead to concerns if we are trying to limit the scope of which microservices can see what kind of data—for example, I might want to limit which microservices can see personally identifiable information (or PII), payment card details, or similar sensitive data. A way to solve this could be to send two different types of events—one that contains PII and can be seen by some mi-

croservices, and another that excludes PII and can be broadcast more widely. This adds complexity in terms of managing visibility of different events and ensuring that both events actually get fired. What happens when a microservice sends the first type of event but dies before the second event can be sent?

Another consideration is that once we put data into an event, it becomes part of our contract with the outside world. We have to be aware that if we remove a field from an event, we may break external parties. Information hiding is still an important concept in event-driven collaboration—the more data we put into an event, the more assumptions external parties will have about the event. My general rule is that I am OK putting information into an event if I'd be happy sharing the same data over a request-response API.

Where to Use It

Event-driven collaboration thrives in situations in which information wants to be broadcast, and in situations in which you are happy to invert intent. Moving away from a model of telling other things what to do and instead letting downstream microservices work this out for themselves has a great deal of attraction.

In a situation in which you are focusing on loose coupling more than other factors, event-driven collaboration is going to have obvious appeal.

The cautionary note is that there are often new sources of complexity that come to the fore with this style of collaboration, especially if you've had limited exposure to it. If you are unsure about this form of communication, remember that our microservice architecture can (and likely will) contain a mix of different styles of interaction. You don't have to go all in with event-driven collaboration; perhaps start with just one event and take it from there.

Personally, I find myself gravitating toward event-driven collaboration almost as a default. My brain seems to have rewired itself in such a way that these types of communication just seem *obvious* to me. This isn't entirely helpful, as it can be tricky to try and explain *why* this is the case, other than to say it *feels* right. But that is just my own built-in bias—I naturally gravitate to what *I* know, based on my own experiences. There is a strong possibility that my attraction to this form of interaction is driven almost entirely by my previous bad experiences with overly coupled systems. I might just be the general fighting the last battle over and over again without considering that perhaps this time it really is different.

What I will say, putting my own biases aside, is that I see far more teams replacing request-response interactions with event-driven interactions than the reverse.

Proceed with Caution

Some of this asynchronous stuff seems fun, right? Event-driven architectures seem to lead to significantly more decoupled, scalable systems. And they can. But these communication styles do lead to an increase in complexity. This isn't just the complexity required to manage publishing and subscribing to messages, as we just discussed, but also complexity in the other problems we might face. For example, when considering long-running async request-response, we have to think about what to do when the response comes back. Does it come back to the same node that initiated the request? If so, what happens if that node is down? If not, do I need to store information somewhere so I can react accordingly? Short-lived async can be easier to manage if you've got the right APIs, but even so, it is a different way of thinking for programmers who are accustomed to intra-process synchronous message calls.

It's time for a cautionary tale. Back in 2006, I was working on building a pricing system for a bank. We would look at market events and work out which items in a portfolio needed to be repriced. Once we determined the list of things to work through, we put these all onto a message queue. We were making use of a grid to create a pool of pricing workers, allowing us to scale up and down the pricing farm on request. These workers used the competing consumers pattern, each one gobbling messages as fast as possible until there was nothing left to process.

The system was up and running, and we were feeling rather smug. One day, though, just after we pushed a release out, we hit a nasty problem: our workers kept dying. And dying. And dying.

Eventually, we tracked down the problem. A bug had crept in whereby a certain type of pricing request would cause a worker to crash. We were using a transacted queue: as the worker died, its lock on the request timed out, and the pricing request was put back on the queue—only for another worker to pick it up and die. This was a classic example of what Martin Fowler calls a catastrophic failover.

Aside from the bug itself, we'd failed to specify a maximum retry limit for the job on the queue. So we fixed the bug, and configured a maximum retry. But we also realized we needed a way to view and potentially replay these bad messages. We ended up having to implement a message hospital (or dead letter queue), where messages got sent if they

failed. We also created a UI to view those messages and retry them if needed. These sorts of problems aren't immediately obvious if you are familiar only with synchronous point-to-point communication.

The associated complexity with event-driven architectures and asynchronous programming in general leads me to believe that you should be cautious in how eagerly you start adopting these ideas. Ensure you have good monitoring in place, and strongly consider the use of correlation IDs, which allow you to trace requests across process boundaries, as we'll cover in depth in Chapter 10.

I also strongly recommend checking out *Enterprise Integration Patterns* by Gregor Hohpe and Bobby Woolf,[4] which contains a lot more detail on the different messaging patterns that you may want to consider in this space.

We also have to be honest, though, about the integration styles that we might consider "simpler"—the problems associated with knowing whether things worked or not is not limited to asynchronous forms of integration. With a synchronous, blocking call, if you get a time-out, did this happen because the request got lost and the downstream party didn't receive it? Or did the request get through, but the response got lost? What do you do in that situation? If you retry, but the original request did get through what then? (Well, this is where idempotency comes in, a topic we cover in Chapter 12.)

Arguably, with regard to failure handling, synchronous blocking calls can cause us just as many headaches when it comes to working out if things have happened (or not). It's just that those headaches may be more familiar to us!

Summary

In this chapter, I broke down some of the key styles of microservice communication and discussed the various trade-offs. There isn't always a single *right* option, but hopefully I've detailed enough information regarding synchronous and asynchronous calls and event-driven and request-response styles of communication to help you make the correct call for your given context. My own biases toward asynchronous, event-driven collaboration are a function not just of my experiences but also of my aversion to coupling in general. But this style of communication comes with significant complexity that cannot be ignored, and every situation is unique.

In this chapter, I briefly mentioned a few specific technologies that can be used to implement these interaction styles. We're now ready to begin the second part of this book—

implementation. In the next chapter, we'll explore implementing microservice communication in more depth.

1 True story.

2 Maarten van Steen and Andrew S. Tanenbaum, *Distributed Systems*, 3rd ed. (Scotts Valley, CA: CreateSpace Independent Publishing Platform, 2017).

3 Please note, this is very simplified—I've completely omitted error-handling code, for example. If you want to know more about async/await, specifically in JavaScript, the Modern JavaScript Tutorial is a great place to start.

4 Gregor Hohpe and Bobby Woolf, *Enterprise Integration Patterns* (Boston: Addison-Wesley, 2003).

Previous chapter
< 3. Splitting the Monolith

Next chapter
II. Implementation >